Hive - Create Table

This chapter explains how to create a table and how to insert data into it. The conventions of creating a table in HIVE is quite similar to creating a table using SQL.

Create Table Statement

Create Table is a statement used to create a table in Hive. The syntax and example are as follows:

Syntax

CREATE [TEMPORARY] [EXTERNAL] TABLE [IF NOT EXISTS] [db_name.] table_name

[(col_name data_type [COMMENT col_comment], ...)] [COMMENT table_comment]
[ROW FORMAT row_format] [STORED AS file_format]

Example

Let us assume you need to create a table named **employee** using **CREATE TABLE** statement. The following table lists the fields and their data types in employee table:

Sr.No	Field Name	Data Type
1	Eid	int
2	Name	String
3	Salary	Float
4	Designation	string

The following data is a Comment, Row formatted fields such as Field terminator, Lines terminator, and Stored File type.

```
COMMENT „Employee details‟
FIELDS TERMINATED BY „\t‟
LINES TERMINATED BY „\n‟
STORED IN TEXT FILE
```

The following query creates a table named **employee** using the above data.

```
hive> CREATE TABLE IF NOT EXISTS employee ( eid int, name String,
> salary String, destination String)
> COMMENT „Employee details‟
> ROW FORMAT DELIMITED
> FIELDS TERMINATED BY „\t‟
> LINES TERMINATED BY „\n‟
> STORED AS TEXTFILE;
```

If you add the option IF NOT EXISTS, Hive ignores the statement in case the table already exists.

On successful creation of table, you get to see the following response:

OK
Time taken: 5.905 seconds hive>

Load Data Statement

Generally, after creating a table in SQL, we can insert data using the Insert statement. But in Hive, we can insert data using the LOAD DATA statement.

While inserting data into Hive, it is better to use LOAD DATA to store bulk records. There are two ways to load data: one is from local file system and second is from Hadoop file system.

Syntax

The syntax for load data is as follows:

LOAD DATA [LOCAL] INPATH 'filepath' [OVERWRITE] INTO TABLE tablename [PARTITION (partcol1=val1, partcol2=val2 ...)]

- LOCAL is identifier to specify the local path. It is optional.
- OVERWRITE is optional to overwrite the data in the table.
- PARTITION is optional.

Example

We will insert the following data into the table. It is a text file named **sample.txt** in **/home/user** directory.

1201	Gopal	45000	Technical manager
1202	Manisha	45000	Proof reader
1203	Masthanvali	40000	Technical writer
1204	Krian	40000	Hr Admin
1205	Kranthi	30000	Op Admin

The following query loads the given text into the table.

hive> LOAD DATA LOCAL INPATH '/home/user/sample.txt' > OVERWRITE INTO TABLE employee;

On successful download, you get to see the following response:

OK
Time taken: 15.905 seconds hive>

Hive - Alter Table

This chapter explains how to alter the attributes of a table such as changing its table name, changing column names, adding columns, and deleting or replacing columns.

Alter Table Statement

It is used to alter a table in Hive.

Syntax

The statement takes any of the following syntaxes based on what attributes we wish to modify in a table.

ALTER TABLE name RENAME TO new_name
ALTER TABLE name ADD COLUMNS (col_spec[, col_spec ...]) ALTER TABLE name DROP [COLUMN] column_name
ALTER TABLE name CHANGE column_name new_name new_type
ALTER TABLE name REPLACE COLUMNS (col_spec[, col_spec ...])

Rename To… Statement

The following query renames the table from **employee** to **emp**.

hive> ALTER TABLE employee RENAME TO emp;

Change Statement

The following table contains the fields of **employee** table and it shows the fields to be changed (in bold).

Field Name	Convert from Data Type	Change Field Name	Convert to Data Type
eid	int	eid	int
name	String	**ename**	String
salary	**Float**	salary	**Double**
designation	String	designation	String

The following queries rename the column name and column data type using the above data:

hive> ALTER TABLE employee CHANGE name ename String; hive> ALTER TABLE employee CHANGE salary salary Double;

System.out.println("Change column successful."); con.close();
}
}

Save the program in a file named HiveAlterChangeColumn.java. Use the following commands to compile and execute this program.

$ javac HiveAlterChangeColumn.java $ java HiveAlterChangeColumn

Output:
Change column successful.

Add Columns Statement

The following query adds a column named dept to the employee table.

```
hive> ALTER TABLE employee ADD COLUMNS (
> dept STRING COMMENT 'Department name');
```

Replace Statement

The following query deletes all the columns from the **employee** table and replaces it with **emp** and **name** columns:

```
hive> ALTER TABLE employee REPLACE COLUMNS (
>       eid INT empid Int,
>       ename STRING name String);
```

Hive - Drop Table

This chapter describes how to drop a table in Hive. When you drop a table from Hive Metastore, it removes the table/column data and their metadata. It can be a normal table (stored in Metastore) or an external table (stored in local file system); Hive treats both in the same manner, irrespective of their types.

Drop Table Statement

The syntax is as follows:

```
DROP TABLE [IF EXISTS] table_name;
```

The following query drops a table named **employee**:

```
hive> DROP TABLE IF EXISTS employee;
```

On successful execution of the query, you get to see the following response:

```
OK
Time taken: 5.3 seconds hive>
```

The following query is used to verify the list of tables:

hive> SHOW TABLES; emp
ok
Time taken: 2.1 seconds hive>

Hive - Partitioning

Hive organizes tables into partitions. It is a way of dividing a table into related parts based on the values of partitioned columns such as date, city, and department. Using partition, it is easy to query a portion of the data.

Tables or partitions are sub-divided into **buckets,** to provide extra structure to the data that may be used for more efficient querying. Bucketing works based on the value of hash function of some column of a table.

For example, a table named **Tab1** contains employee data such as id, name, dept, and yoj (i.e., year of joining). Suppose you need to retrieve the details of all employees who joined in 2012. A query searches the whole table for the required information. However, if you partition the employee data with the year and store it in a separate file, it reduces the query processing time. The following example shows how to partition a file and its data:

The following file contains employeedata table.

/tab1/employeedata/file1

id, name, dept, yoj

1, gopal, TP, 2012

2, kiran, HR, 2012

3, kaleel,SC, 2013

4, Prasanth, SC, 2013

The above data is partitioned into two files using year.

/tab1/employeedata/2012/file2

1, gopal, TP, 2012

2, kiran, HR, 2012

/tab1/employeedata/2013/file3

3, kaleel,SC, 2013

4, Prasanth, SC, 2013

Adding a Partition

We can add partitions to a table by altering the table. Let us assume we have a table called **employee** with fields such as Id, Name, Salary, Designation, Dept, and yoj.

Syntax:

ALTER TABLE table_name ADD [IF NOT EXISTS] PARTITION partition_spec [LOCATION 'location1'] partition_spec [LOCATION 'location2'] ...;

partition_spec:
: (p_column = p_col_value, p_column = p_col_value, ...)

The following query is used to add a partition to the employee table.

hive> ALTER TABLE employee
> ADD PARTITION (year=‟2013‟)
> location '/2012/part2012';

Renaming a Partition

The syntax of this command is as follows.

ALTER TABLE table_name PARTITION partition_spec RENAME TO PARTITION partition_spec;

The following query is used to rename a partition:
hive> ALTER TABLE employee PARTITION (year=‟1203‟) > RENAME TO PARTITION (Yoj=‟1203‟);

Dropping a Partition

The following syntax is used to drop a partition:

ALTER TABLE table_name DROP [IF EXISTS] PARTITION partition_spec, PARTITION partition_spec,...;

The following query is used to drop a partition:

hive> ALTER TABLE employee DROP [IF EXISTS] > PARTITION (year=‟1203‟);

Hiveql Select...Where

The Hive Query Language (HiveQL) is a query language for Hive to process and analyze structured data in a Metastore. This chapter explains how to use the SELECT statement with WHERE clause.

SELECT statement is used to retrieve the data from a table. WHERE clause works similar to a condition. It filters the data using the condition and gives you a finite result. The built-in operators and functions generate an expression, which fulfils the condition.

Syntax

Given below is the syntax of the SELECT query:

```
SELECT [ALL | DISTINCT] select_expr, select_expr, ...
FROM table_reference [WHERE where_condition] [GROUP BY col_list] [HAVING having_condition]
[CLUSTER BY col_list | [DISTRIBUTE BY col_list] [SORT BY col_list]] [LIMIT number];
```

Example

Let us take an example for SELECT…WHERE clause. Assume we hav given below, with fields named Id, Name, Salary, Designation, and Dept. Generate a query to
retrieve the employee details who earn a salary of more than Rs 30000.

```
+------ +-------------- +------------ +------------------- +-------- +
| ID    | Name         | Salary      | Designation       | Dept     |
+------ +-------------- +------------ +------------------- +-------- +
|1201   | Gopal        | 45000       | Technical manager | TP       |
|1202   | Manisha      | 45000       | Proofreader       | PR       |
|1203   | Masthanvali  | 40000       | Technical writer  | TP       |
|1204   | Krian        | 40000       | Hr Admin          | HR       |
|1205   | Kranthi      | 30000       | Op Admin          | Admin    |
+------ +-------------- +------------ +------------------- +-------- +
```

The following query retrieves the employee details using the above scenario:

```
hive> SELECT * FROM employee WHERE salary>30000;
```

On successful execution of the query, you get to see the following response:

```
+------+--------------+-------------+-------------------+--------+

| ID    | Name         | Salary      | Designation       | Dept     |
+------ +-------------- +------------ +------------------- +-------- +
|1201   | Gopal        | 45000       | Technical manager | TP       |
|1202   | Manisha      | 45000       | Proofreader       | PR       |
|1203   | Masthanvali  | 40000       | Technical writer  | TP       |
|1204   | Krian        | 40000       | Hr Admin          | HR       |
+------ +-------------- +------------ +------------------- +-------- +
```

Hiveql Select...Order By

This chapter explains how to use the ORDER BY clause in a SELECT statement. The ORDER BY clause is used to retrieve the details based on one column and sort the result set by ascending or descending order.

Syntax

Given below is the syntax of the ORDER BY clause:

SELECT [ALL | DISTINCT] select_expr, select_expr, ...
FROM table_reference [WHERE where_condition] [GROUP BY col_list] [HAVING having_condition] [ORDER BY col_list]] [LIMIT number];

Example

Let us take an example for SELECT...ORDER BY clause. Assume employee table as given below, with the fields named Id, Name, Salary, Designation, and Dept. Generate a query to retrieve the employee details in order by using Department name.

ID	Name	Salary	Designation	Dept
1201	Gopal	45000	Technical manager	TP
1202	Manisha	45000	Proofreader	PR
1203	Masthanvali	40000	Technical writer	TP
1204	Krian	40000	Hr Admin	HR
1205	Kranthi	30000	Op Admin	Admin

The following query retrieves the employee details using the above scenario:

hive> SELECT Id, Name, Dept FROM employee ORDER BY DEPT;

On successful execution of the query, you get to see the following response:

ID	Name	Salary	Designation	Dept
1205	Kranthi	30000	Op Admin	Admin
1204	Krian	40000	Hr Admin	HR
1202	Manisha	45000	Proofreader	PR
1201	Gopal	45000	Technical manager	TP
1203	Masthanvali	40000	Technical writer	TP

Hiveql Group By

This chapter explains the details of GROUP BY clause in a SELECT statement. The GROUP BY clause is used to group all the records in a result set using a particular collection column. It is used to query a group of records.

Syntax

The syntax of GROUP BY clause is as follows:

SELECT [ALL | DISTINCT] select_expr, select_expr, ...
FROM table_reference [WHERE where_condition] [GROUP BY col_list] [HAVING having_condition] [ORDER BY col_list]] [LIMIT number];

Example

Let us take an example of SELECTeetable…GROUPasgiven BY cla below, with Id, Name,

Salary, Designation, and Dept fields. Generate a query to retrieve the number of employees in each department.

ID	Name	Salary	Designation	Dept
1201	Gopal	45000	Technical manager	TP
1202	Manisha	45000	Proofreader	PR
1203	Masthanvali	40000	Technical writer	TP
1204	Krian	45000	Proofreader	PR
1205	Kranthi	30000	Op Admin	Admin

The following query retrieves the employee details using the above scenario.

```
hive> SELECT Dept,count(*) FROM employee GROUP BY DEPT;
```

On successful execution of the query, you get to see the following response:

Dept	Count(*)
Admin	
PR	
TP	

Hiveql Joins

JOINS is a clause that is used for combining specific fields from two tables by using values common to each one. It is used to combine records from two or more tables in the database. It is more or less similar to SQL JOINS.

Syntax

join_table:

```
table_reference JOIN table_factor [join_condition]
| table_reference {LEFT|RIGHT|FULL} [OUTER] JOIN table_reference join_condition
| table_reference LEFT SEMI JOIN table_reference join_condition
| table_reference CROSS JOIN table_reference [join_condition]
```

Example

We will use the following two tables in this chapter. Consider the following table named CUSTOMERS..

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00

5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

Consider another table ORDERS as follows:

OID	DATE	CUSTOMER_ID	AMOUNT
102	2009-10-08 00:00:00		3000
100	2009-10-08 00:00:00		1500
101	2009-11-20 00:00:00		1560
103	2008-05-20 00:00:00		2060

There are different types of joins given as follows:

- JOIN
- LEFT OUTER JOIN
- RIGHT OUTER JOIN
- FULL OUTER JOIN

JOIN

JOIN clause is used to combine and retrieve the records from multiple tables. JOIN is same as OUTER JOIN in SQL. A JOIN condition is to be raised using the primary keys and foreign keys of the tables.

The following query executes JOIN on the CUSTOMER and ORDER tables, and retrieves the records:

```
hive> SELECT c.ID, c.NAME, c.AGE, o.AMOUNT
>     FROM CUSTOMERS c JOIN ORDERS o
>     ON (c.ID = o.CUSTOMER_ID);
```

On successful execution of the query, you get to see the following response:

ID	NAME	AGE	AMOUNT
3	kaushik	23	3000
3	kaushik	23	1500
2	Khilan	25	1560
4	Chaitali	25	2060

LEFT OUTER JOIN

The HiveQL LEFT OUTER JOIN returns all the rows from the left table, even if there are no matches in the right table. This means, if the ON clause matches 0 (zero) records in the right table, the JOIN still returns a row in the result, but with NULL in each column from the right table.

A LEFT JOIN returns all the values from the left table, plus the matched values from the right table, or NULL in case of no matching JOIN predicate.

The following query demonstrates LEFT OUTER JOIN between CUSTOMER and ORDER tables:

```
hive> SELECT c.ID, c.NAME, o.AMOUNT, o.DATE
>     FROM CUSTOMERS c
>     LEFT OUTER JOIN ORDERS o
>     ON (c.ID = o.CUSTOMER_ID);
```

On successful execution of the query, you get to see the following response:

```
+----  +----------     +--------    +--------------------          +
| ID | NAME          | AMOUNT | DATE                          |
+----  +----------     +--------    +--------------------          +
| 1     | Ramesh      | NULL     | NULL                          |
| 2     | Khilan      | 1560     | 2009-11-20 00:00:00 |
| 3     | kaushik     | 3000     | 2009-10-08 00:00:00 |
| 3     | kaushik     | 1500     | 2009-10-08 00:00:00 |
| 4     | Chaitali    | 2060     | 2008-05-20 00:00:00 |
| 5     | Hardik      | NULL     | NULL                          |
| 6     | Komal       | NULL     | NULL                          |
| 7     | Muffy       | NULL     | NULL                          |
+----  +----------     +--------    +--------------------          +
```

RIGHT OUTER JOIN

The HiveQL RIGHT OUTER JOIN returns all the rows from the right table, even if there are no matches in the left table. If the ON clause matches 0 (zero) records in the left table, the JOIN still returns a row in the result, but with NULL in each column from the left table.

A RIGHT JOIN returns all the values from the right table, plus the matched values from the left table, or NULL in case of no matching join predicate.

The following query demonstrates RIGHT OUTER JOIN between the CUSTOMER and ORDER tables.

```
hive> SELECT c.ID, c.NAME, o.AMOUNT, o.DATE
>     FROM CUSTOMERS c
>     RIGHT OUTER JOIN ORDERS o
>     ON (c.ID = o.CUSTOMER_ID);
```

On successful execution of the query, you get to see the following response:

```
+------   +----------     +--------    +--------------------          +
| ID       | NAME        | AMOUNT | DATE                          |
+------   +----------     +--------    +--------------------          +
| 3        | kaushik     | 3000     | 2009-10-08 00:00:00 |
| 3        | kaushik     | 1500     | 2009-10-08 00:00:00 |
| 2        | Khilan      | 1560     | 2009-11-20 00:00:00 |
| 4        | Chaitali | 2060        | 2008-05-20 00:00:00 |
```

```
+------   +----------      +--------     +--------------------          +
```

FULL OUTER JOIN

The HiveQL FULL OUTER JOIN combines the records of both the left and the right outer tables that fulfil the JOIN condition. The joined table contains either all the records from both the tables, or fills in NULL values for missing matches on either side.

The following query demonstrates FULL OUTER JOIN between CUSTOMER and ORDER tables:

```
hive> SELECT c.ID, c.NAME, o.AMOUNT, o.DATE
>       FROM CUSTOMERS c
>       FULL OUTER JOIN ORDERS o
>       ON (c.ID = o.CUSTOMER_ID);
```

On successful execution of the query, you get to see the following response:

```
+------   +----------      +--------     +--------------------          +
| ID      | NAME          | AMOUNT | DATE                              |
+------   +----------      +--------     +--------------------          +
| 1       | Ramesh        | NULL   | NULL                              |
| 2       | Khilan        | 1560   | 2009-11-20 00:00:00 |
| 3       | kaushik       | 3000   | 2009-10-08 00:00:00 |
| 3       | kaushik       | 1500   | 2009-10-08 00:00:00 |
| 4       | Chaitali      | 2060   | 2008-05-20 00:00:00 |
| 5       | Hardik        | NULL   | NULL                              |
| 6       | Komal         | NULL   | NULL                              |
| 7       | Muffy         | NULL   | NULL                              |
| 3       | kaushik       | 3000   | 2009-10-08 00:00:00 |
| 3       | kaushik       | 1500   | 2009-10-08 00:00:00 |
| 2       | Khilan        | 1560   | 2009-11-20 00:00:00 |
                                                  2008-05          -20
|     4   |          Chaitali |     2060|                 00:00:00 |
+     ------+          ----------+     --------+                 ---------------------+
```

Hive - View and Indexes

This chapter describes how to create and manage views. Views are generated based on user requirements. You can save any result set data as a view. The usage of view in Hive is same as that of the view in SQL. It is a standard RDBMS concept. We can execute all DML operations on a view.

Creating a View

You can create a view at the time of executing a SELECT statement. The syntax is as follows:

```
CREATE VIEW [IF NOT EXISTS] view_name [(column_name [COMMENT column_comment],
...) ]
[COMMENT table_comment] AS SELECT ...
```

Example

Let us take an example for view. Assume employee table as given below, with the fields Id, Name, Salary, Designation, and Dept. Generate a query to retrieve the employee details who earn a salary of more than Rs 30000. We store the result in a view named **emp_30000.**

ID	Name	Salary	Designation	Dept
1201	Gopal	45000	Technical manager	TP
1202	Manisha	45000	Proofreader	PR
1203	Masthanvali	40000	Technical writer	TP
1204	Krian	40000	Hr Admin	HR
1205	Kranthi	30000	Op Admin	Admin

The following query retrieves the employee details using the above scenario:

```
hive> CREATE VIEW emp_30000 AS
>       SELECT * FROM employee
>       WHERE salary>30000;
```

Dropping a View

Use the following syntax to drop a view:

```
DROP VIEW view_name
```

The following query drops a view named as emp_30000:

```
hive> DROP VIEW emp_30000;
```

Creating an Index

An Index is nothing but a pointer on a particular column of a table. Creating an index means creating a pointer on a particular column of a table. Its syntax is as follows:

```
CREATE INDEX index_name
ON TABLE base_table_name (col_name, ...) AS 'index.handler.class.name'
[WITH DEFERRED REBUILD]
[IDXPROPERTIES (property_name=property_value, ...)] [IN TABLE index_table_name]
[PARTITIONED BY (col_name, ...)]
[
[ ROW FORMAT ...] STORED AS ...
| STORED BY ...
]
[LOCATION hdfs_path] [TBLPROPERTIES (...)]
```

Example

Let us take an example for index. Use the same employee table that we have used earlier with the

fields Id, Name, Salary, Designation, and Dept. Create an index named index_salary on the salary column of the employee table.

The following query creates an index:

```
hive> CREATE INDEX inedx_salary ON TABLE employee(salary)
> AS 'org.apache.hadoop.hive.ql.index.compact.CompactIndexHandler';
```

It is a pointer to the salary column. If the column is modified, the changes are stored using an index value.

Dropping an Index

The following syntax is used to drop an index:

```
DROP INDEX <index_name> ON <table_name>
```

The following query drops an index named index_salary:

```
hive> DROP INDEX index_salary ON employee;
```